Transformer mreže

Priprema za odbranu projekta „klasifikacija oglasa po naslovu“ · sve je vezano za moj kod (`src/model.py`) i moje rezultate

> **Najkraće moguće (ako te neko pita u liftu):** Transformer je neuronska mreža koja svaku reč u naslovu „pogleda“ u odnosu na sve ostale reči (mehanizam *pažnje*), pa vektor svake reči postaje zavistan od konteksta. Zatim se vektori svih reči uproseče i iz tog proseka se predviđa jedna od 10 grupa. Moj model je mali i treniran od nule – nije GPT, BERT niti bilo šta pretrenirano.

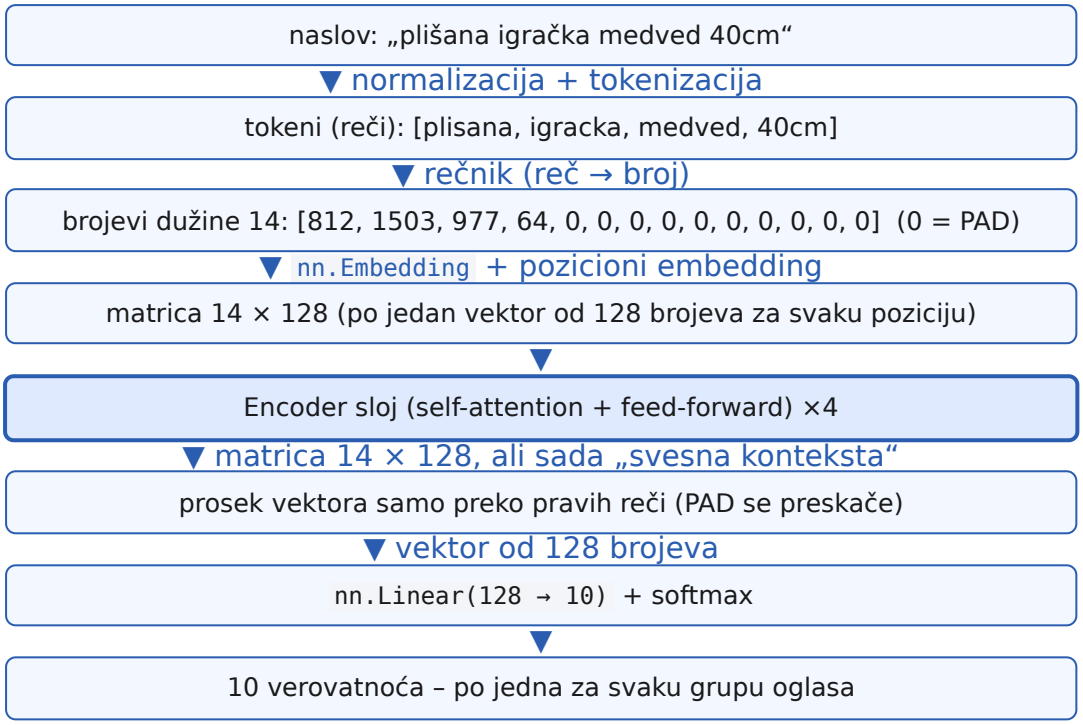
1. Šta je problem koji transformer rešava

Tekst je **niz reči čiji redosled i kontekst menjaju značenje**. Prethodni projekat (MLP nad TF-IDF) gleda naslov kao „kesu reči“: zna koje reči postoje, ali ne i kako se one odnose jedna prema drugoj. Primer: reč „*kuhinja*“ u „kuhinja za lutke“ i „kuhinja Mebla“ ima različito značenje (igračke vs nameštaj), a kontekst (druge reči) to otkriva.

Pre transformera su se za nizove koristile **RNN/LSTM** mreže: čitaju reč po reč, redom, i pamte „stanje“. Problemi: sporo (ne može paralelno, jer reč 5 čeka reč 4) i teško pamte daleke veze. Transformer (rad *„Attention Is All You Need“*, Vaswani i saradnici, 2017) to rešava tako što **sve reči obrađuje istovremeno**, a veze između reči računa direktno pomoću pažnje.

Današnji GPT, BERT, Claude itd. su svi transformeri – samo ogromni i pretrenirani na tekstu sa interneta. Moj je mali, **samo „encoder“ deo** (čita tekst i pravi njegovu reprezentaciju; „decoder“ služi za generisanje teksta i meni ne treba).

2. Ceo moj model na jednoj slici



Brojevi su iz moje konačne mreže: `d_model=128, n_heads=4, n_layers=4, d_ff=256, dropout=0.3`, rečnik 4204 reči, ukupno **1.071.370** parametara.

3. Korak po korak

3.1 Tokenizacija i rečnik

Mreža radi sa brojevima, ne sa rečima. Zato svaka reč dobija broj. Rečnik pravim samo iz **trening** skupa (da ništa iz testa ne „procuri“), i uzimam samo reči koje se pojavljuju bar `min_count=5` puta. Rezervisani brojevi: **0 = PAD** (popuna), **1 = UNK** (nepoznata reč). Naslovi se dopunjavaju/seku na 14 reči da bi svi imali istu dužinu i da bi mogli da se slažu u matricu (batch).

Zašto normalizacija pre toga: bez nje su „Plišana“, „plisana“ i „PLISANA“ tri različita tokena. To sam izmerio: validacioni gubitak 0.187 (bez) → 0.131 (puna normalizacija).

3.2 Embedding

`nn.Embedding(4206, 128)` je obična tabela: red broj *k* je vektor od 128 brojeva za reč sa brojem *k*. Na početku su vrednosti nasumične, a **treningom se uče** tako da reči sličnog značenja (npr. „lutka“ i „igračka“) dobiju slične vektore. To je ono što MLP nema: on svaku reč tretira kao nezavisnu kolonu.

3.3 Pozicioni embedding

Pažnja (sledeći korak) ne zna za redosled – „pas ujeo čoveka“ i „čovek ujeo psa“ bi izgledali isto. Zato svakoj poziciji (0–13) dodajem još jedan naučeni vektor: `h = emb(reči) +`

`pos(pozicije)`. Sabiranjem se u istom vektoru nalaze i „koja je reč“ i „gde stoji“. (Original koristi fiksne sinus/kosinus pozicije; ja koristim naučene, što je jednostavnije i radi podjednako dobro za kratke nizove.)

3.4 Self-attention (srce transformera)

Ideja: da bi se razumela jedna reč, treba je uporediti sa svim ostalim rečima i uzeti više informacija od onih koje su joj bitne. Svaka reč iz svog vektora x pravi tri nova vektora pomoću tri naučene matrice:

- **Q (query, „upit“)** = $x \cdot W_Q$ – „šta ja tražim?“
- **K (key, „ključ“)** = $x \cdot W_K$ – „šta ja nudim / po čemu me prepoznaju?“
- **V (value, „vrednost“)** = $x \cdot W_V$ – „koju informaciju prenosim ako me izaberu“

Formula:

```
Attention(Q, K, V) = softmax( Q · Kᵀ / √d_k ) · V
```

1. **Q·Kᵀ**: skalarni proizvod upita svake reči sa ključem svake reči → matrica „koliko je reč i bitna za reč j“ (veličine 14 × 14). Veći proizvod = sličniji vektori.
2. **/ √d_k**: skaliranje. Sa velikim dimenzijama proizvodi postanu veliki, softmax se „zaglavi“ (jedna težina ≈1, ostale ≈0) i gradijenti nestaju. Deljenje to stabilizuje.
3. **softmax** po redu: pretvara ocene u težine koje su ≥0 i sabiraju se na 1.
4. **· V**: nova reprezentacija reči = *težinski prosek* vrednosti svih reči.

Mali brojčani primer (3 reči, $d_k = 4$, pa je $\sqrt{d_k} = 2$). Neka je upit prve reči sa ključevima sve tri reči dao ocene [4, 2, 0].
Skaliranje: [4, 2, 0] / 2 = [2, 1, 0].
Softmax: $e^2 = 7.39$, $e^1 = 2.72$, $e^0 = 1$ → zbir 11.11 → težine **[0.665, 0.245, 0.090]**.
Neka su vrednosti v_1=[1,0], v_2=[0,1], v_3=[1,1]. Izlaz za prvu reč: 0.665·[1,0] + 0.245·[0,1] + 0.090·[1,1] = **[0.755, 0.335]**.
Dakle prva reč je pretežno „uzela“ informaciju od sebe (težina 0.665), malo od druge, još manje od treće.

Važno: **W_Q, W_K, W_V se uče** – mreža sama nauči šta je „bitno“ za klasifikaciju oglasa. Zato se kaže *self*-attention: Q, K i V potiču iz istog niza.

3.5 Multi-head (više glava)

Umesto jedne pažnje, radim **4 paralelne** (`n_heads=4`), svaka na delu vektora (128 / 4 = 32 dimenzije). Razlog: svaka glava može da nauči drugu vrstu veze (jedna npr. „pridev uz imenicu“, druga „broj uz jedinicu mere“). Izlazi glava se nalepe (4×32 = 128) i propuste kroz još jednu linearnu matricu.

3.6 Feed-forward, residual i LayerNorm

Jedan encoder sloj nije samo pažnja:

- **Feed-forward**: za svaku reč posebno mali MLP `128 → 256 → ReLU → 128`. Pažnja meša informacije *između* reči, feed-forward ih dodatno obrađuje *unutar* svake reči.
- **Residual (skip) veza**: izlaz = ulaz + F(ulaz). Mreža uči samo „ispravku“; ovo omogućava da se slojevi slažu duboko (4 sloja) i da gradijent lepo teče unazad.
- **LayerNorm**: normalizuje vektor svake reči (srednja vrednost 0, odstupanje 1) i stabilizuje trening.
- **Dropout** (0.3): u treningu nasumično „ugasi“ 30% veza da mreža ne bi samo pamtila trening skup.

```
x → [ self-attention ] → + x → LayerNorm → [ feed-forward ] → + → LayerNorm → izlaz
        (residual)                          (residual)
```

Ovaj sloj se ponovi 4 puta (`n_layers=4`); sa svakim slojem reprezentacija reči je sve „kontekstualnija“. U PyTorch-u je sve to jedan red: `nn.TransformerEncoderLayer`, a slaganje radi `nn.TransformerEncoder`.

3.7 Maska za PAD

Naslovi su kratki, a vektor je dug 14, pa su mnoge pozicije PAD. `src_key_padding_mask` kaže pažnji da PAD pozicije **ignoriše** (ocena $-\infty$ → težina 0), da prazne pozicije ne bi uticale na prave reči.

3.8 Pooling i izlaz

Posle encodera imam po jedan vektor za svaku od 14 pozicija, a treba mi **jedan vektor po naslovu**. Zato računam prosek vektora samo preko pravih reči (PAD ne ulazi). Taj vektor ide u `nn.Linear(128, 10)` koji daje 10 ocena (logita); softmax ih pretvara u verovatnoće grupa.

3.9 Funkcija greške i trening

- **Cross-entropy**: `loss = −log(verovatnoća tačne grupe)`. Ako mreža tačnoj grupi da 0.99, greška je mala (0.01); ako joj da 0.1, velika (2.3). Za razliku od tačnosti, kažnjava i „nesigurne“ tačne odgovore, i ima gradijent po kome se uči.
- **Backpropagation**: `loss.backward()` – PyTorch (autograd) sam računa gradijent greške po svakoj težini pravilom lančanog izvoda. Ručno bih to morao da izvedem za pažnju, LayerNorm… što je glomazno; ideja je ista kao u prvom projektu.
- **AdamW** (lr 1e-3, weight decay 0.01): Adam sa adaptivnim korakom po težini + L2-slično „smanjivanje“ težina (regularizacija) odvojeno od gradijenta.
- **Mini-batch 128**: težine se pomere nakon svakih 128 naslova (kao i u MLP-u).
- **Epoha** = jedan prolaz kroz ceo trening skup. **Early stopping**: ako validacioni gubitak ne padne 3 epohe zaredom, stajem i uzimam najbolje težine. Tako validacioni skup bira trenutak prestanka, a test se ne dira.

4. Koliko parametara i odakle

deo	račun	parametara
embedding reči	4206 × 128	538.368
pozicioni embedding	14 × 128	1.792
jedan encoder sloj	pažnja ≈ 4·128² + bias (66.048) + feed-forward (65.920) + 2 LayerNorm (512)	132.480
4 encoder sloja	4 × 132.480	529.920
izlazni sloj	128 × 10 + 10	1.290
ukupno		**1.071.370**

Pola parametara je u embeddingu (tabela reči), pola u slojevima. Ovo je sitnica prema GPT-u (milijarde), ali dovoljno za 10 grupa i kratke naslove.

5. Moji rezultati i šta znače

model	test tačnost	macro F1
MLP nad TF-IDF (1. projekat)	**99.30%**	0.9926
Transformer (ovaj projekat)	97.57%	0.9748

- **Normalizacija pomaže** (val. gubitak 0.187 → 0.131), svaki korak (mala slova, interpunkcija, puna) poboljšava.
- **Veličina mreže skoro nije bitna**: mreža sa 144k parametara je jednako dobra kao ona sa 1.07M; razlike su u okviru šuma jednog treniranja.
- **Zašto je transformer lošiji od MLP-a?** Na naslovima bez nepoznatih reči tačnost mu je 99.30% (isto kao MLP). Ali 55% test naslova ima bar jednu reč koja nije u rečniku (UNK; tačnost tamo 96.15%) i 87% svih grešaka sadrži takvu reč. TF-IDF sa n-gramima *slova* prepoznaje delove nepoznatih reči i tipfelere, a rečnik reči sve to svodi na isti UNK. To je **korelacija**; drugu tokenizaciju (podreči) nisam probao da bih potvrdio uzrok.
- **Zašto ni kontekst nije pomogao:** naslovi su kratki, a grupa se uglavnom prepoznaje po jednoj-dve ključne reči. Za to je „kesa reči“ već skoro dovoljna, pa pažnja nema šta mnogo da doda.

Poštene napomene (reći ih sam pre nego što te pitaju): jedan seed po konfiguraciji; `min_count=5` i dropout 0.3 su na ivici ispitanog opsega; provera sa pomešanim labelama pokazuje da nema curenja podataka, ali ne dokazuje da mreža može da pamti.

6. Transformer vs MLP – razlike

	MLP + TF-IDF	Transformer
ulaz	vektor TF-IDF težina (69.566 brojeva), redosled se gubi	niz tokena, pozicija se čuva
reč	jedna kolona, nezavisna od ostalih	naučeni vektor (embedding) koji zavisi od konteksta
nepoznate reči	n-grami slova ih delimično hvataju	UNK (u mom rečniku)
treniranje	brzo	sporije
kad je bolji	kratki tekstovi, ključne reči, malo podataka	duži tekstovi, značenje zavisi od konteksta, mnogo podataka (pretrenirani)

7. Pitanja koja mogu da dobiješ na odbrani

Šta je pažnja (attention) svojim rečima?

Mehanizam kojim svaka reč izračuna koliko su joj bitne ostale reči i napravi novi vektor kao težinski prosek njihovih informacija. Težine se računaju iz sličnosti upita (Q) i ključeva (K), a prosek se pravi nad vrednostima (V).

Zašto deliš sa $\sqrt{d_k}$?

Skalarni proizvodi rastu sa dimenzijom, pa softmax postane „oštar“ (skoro 0/1) i gradijent skoro nestane. Deljenje drži vrednosti u razumnom opsegu.

Čemu služi pozicioni embedding?

Pažnja nema pojma o redosledu (tretira reči kao skup). Pozicioni vektor se sabira sa vektorom reči da bi mreža znala gde reč stoji.

Čemu multi-head?

Svaka glava gleda drugi podprostor i može da uči drugu vrstu veze između reči; rezultati se spajaju.

Šta rade residual veze i LayerNorm?

Residual (x + F(x)) olakšava trening dubljih mreža i protok gradijenta. LayerNorm normalizuje vektor svake reči i stabilizuje trening.

Šta je PAD i zašto maska?

PAD je popuna da svi naslovi budu dužine 14. Maska sprečava da pažnja i prosek koriste te prazne pozicije.

Zašto prosek vektora na kraju, a ne npr. samo prva reč?

Grupa oglasa zavisi od celog naslova, a prosek je najjednostavniji način da se niz vektora svede na jedan. (BERT koristi poseban [CLS] token; prosek je jednostavna i često podjednako dobra alternativa.)

Da li si koristio pretrenirani model (BERT)?

Ne. Mreža je trenirana od nule na ~oglasima iz baze, samo sa PyTorch slojevima. Pretrenirani bi verovatno bio bolji, ali je zadatak bio da se prođe ceo postupak sam. (To je pretpostavka, nisam probao.)

Zašto je transformer lošiji od MLP-a?

Verovatno zbog rečnika: 55% naslova ima bar jednu nepoznatu reč, a 87% grešaka sadrži takvu reč; na naslovima bez nepoznatih reči tačnost je 99.30%, kao MLP. Ne mogu da tvrdim uzrok sigurno jer nisam probao tokenizaciju na podreči, to je sledeći korak. Plus kratki naslovi ne koriste prednost transformera (kontekst).

Kako si birao hiperparametre?

Na validacionom skupu, po jedan parametar odjednom (normalizacija, min_count, veličina, dropout), po validacionom gubitku. Test skup je korišćen samo jednom, na kraju.

Ograničenja: jedan seed, dve vrednosti na ivici opsega.

Šta je backpropagation, jesi li ga pisao?

Računanje gradijenta greške po svim težinama unazad kroz mrežu pomoću lančanog izvoda. U PyTorch-u to radi `loss.backward()` (autograd). Princip znam, ali ga nisam ručno implementirao za transformer.

Zašto cross-entropy, a ne samo tačnost?

Tačnost nije glatka (nema gradijent) i ne razlikuje sigurnog od nesigurnog tačnog odgovora. Cross-entropy kažnjava −log(p tačne klase), diferencijabilna je i mere „koliko je mreža sigurna“.

Kako znaš da se nije preprilagodila?

Poredim gubitak i tačnost na train/val/test (razlika je mala), koristim dropout, weight decay i early stopping, a kontrola sa pomešanim labelama pokazuje da nema curenja.

8. Rečnik pojmova

pojam	značenje
token	jedinica teksta (kod mene reč)
embedding	naučeni vektor brojeva za token
d_model	dužina vektora po tokenu (128)
Q, K, V	upit, ključ, vrednost – tri projekcije istog vektora
self-attention	pažnja gde Q, K i V potiču iz istog niza
head	jedna od paralelnih pažnji (4)
encoder	deo transformera koji čita ulaz (decoder generiše izlaz – ne koristim ga)
residual	skip veza: x + F(x)
LayerNorm	normalizacija vektora jedne reči
logit	sirova ocena klase pre softmax-a
softmax	pretvara ocene u verovatnoće koje se sabiraju na 1
cross-entropy	−log(p tačne klase)
epoha / batch	jedan prolaz kroz podatke / grupa od 128 naslova
dropout	nasumično gašenje veza u treningu (regularizacija)
early stopping	prekid kad validacioni gubitak prestane da pada
UNK / PAD	nepoznata reč / popuna

Izvor ideje: Vaswani et al., „Attention Is All You Need“, 2017. Brojevi rezultata su iz mojih sveski 02 i 03.